Defects.

Also known as Bugs!

Detect product issues before users do

Óscar Barrios
Test Automation Specialist

1. Terminology of a defect
2. Bug type
3. Severity of a bug
4. Priority of a bug
5. Bug life cycle
6. Bug tracking tools

Defect or bug

When the behavior of an application is different than expected, we will have a defect.

Any deviation from what is mentioned in the functional specifications can be considered a defect, for instance, it can be a different result or a different user interface.

Error or mistake

Problems in either a software design or source code that arise a bug.

As an example, if clicking a button, it doesn't trigger an expected action, we might have an error in the code that involves this button.

Crash

Occurs when an application stops working and exits.

Often you can obtain crash logs that includes the exact state details before the application crashes, it can be useful to reproduce the issue and find the error in the code.

Debugging

The process of finding the reason for an error.

Usually, we run "step by step" the application, checking the state of the variables in our code, the flow between methods, so we can discover the issue and fix the bug.

Functional

Functional defects are these cases where the behavior of software is not compliant with the functional requirements.

For example, a user fill a form and click the button to submit it, but this button doesn't trigger any action.

Usability

Usability defects make an application inconvenient to use, we call this a poor user experience. A user interface that is difficult to navigate, that has too many clicks on its workflow, or it doesn't guide you in a natual manner, can be considered usability bugs.

To identify such defects, we can make use of Web Content Accessibility Guidelines (WCAG).

Performance

Performance defects are those bound to software's speed, stability, response time, and resource consumption, and are discovered during performance testing.

An example of a performance defect is a system's response time being X times longer than that stated in the requirements.

Security

Security bugs make your project vulnerable. This bug opens your software, your company, and your clients up to a serious potential attack. These attacks can be costly if they cause data breaches.

The Open Web Application Security Project (OWASP) lists ten security risks, but some of the most common are encryption errors, susceptibility to SQL injections, XSS vulnerabilities, buffer overflows, logical errors, and inferior authentication.

Compatibility

A compatibility bug could show an unexpected behavior running an application in a different operating systems or using different hardware.

The defects can be found on the UI as changes in font size, content alignment, scroll bar, and so on. But they can also be caused to specific hardware communication or other application dependencies.

Related to this kind of bug, we also can have a **backwards-compatibility** issue, which causes that a new version of the application lose the interoperability with other systems.

Others

There are many others kind of errors that can be considered a bug. Most of them are errors that can be detected during compilation or running unit tests, so they might never be found in production by the final user.

Some examples are syntax errors, unit-level bugs, system-level integration bugs, out of bounds bugs, code duplication and data type mistmatch.

Severity of a bug

Bug severity in testing is a degree of impact a bug has on the software application. They used to be categorized as critical, major, normal, minor and enhancement/cosmetic. This value can be set by the user detecting the bug.

Critical: This defect indicates complete shut-down of the process, nothing can proceed further

Major: It is a highly severe defect and collapses the system. However, certain parts of the system remain functional

Normal: It causes some undesirable behavior, but the system is still functional

Minor: It won't cause any major break-down of the system

Enhancement: It can be considered as an improvement, instead of a bug.

4 Priority of a bug

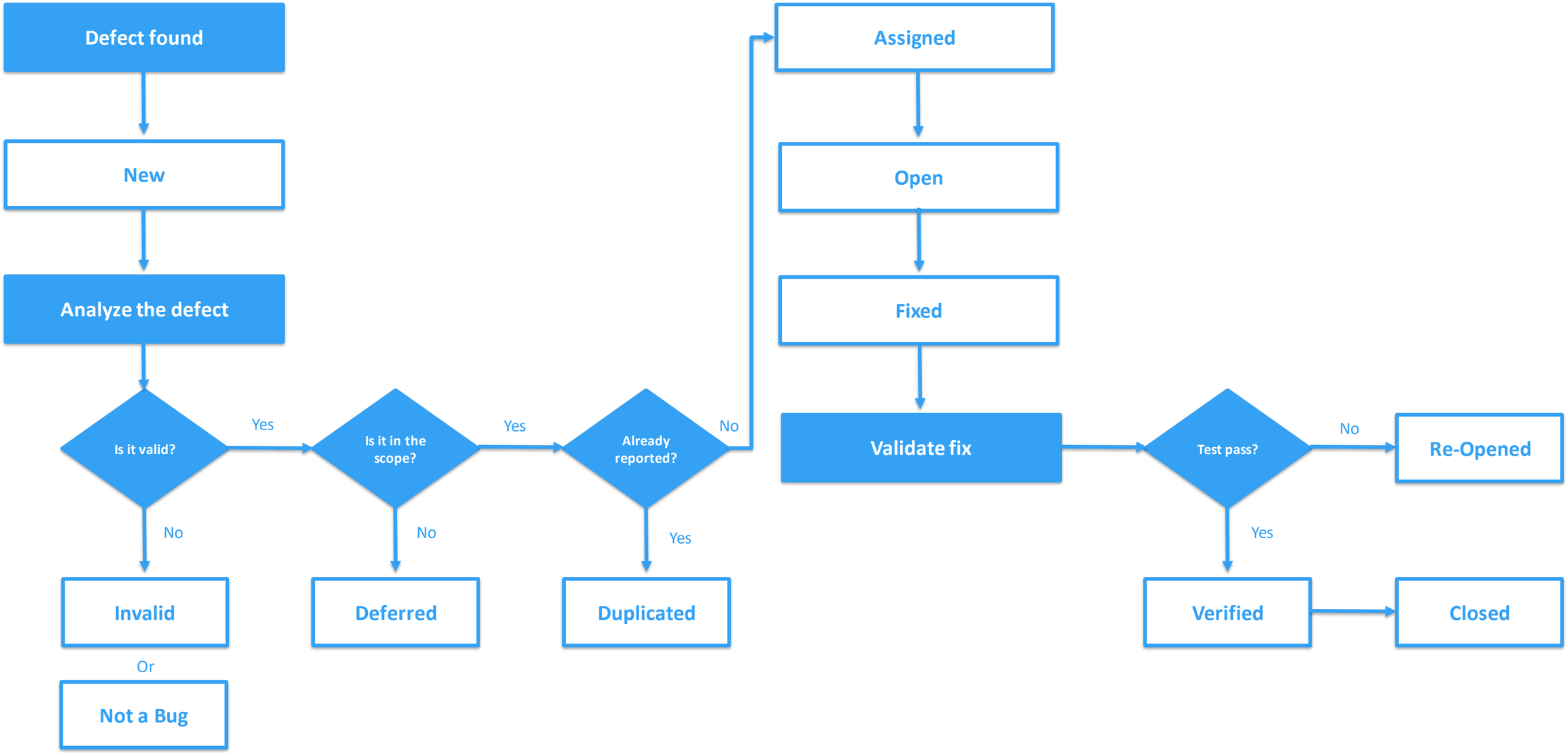
Priority is defined as the order in which the bugs should be fixed. A bug that leaves the system unusable has a higher priority.
Priority is driven by business value while Severity is driven by functionality, so the priority is decided in consultation with the manager or client.

Low: Bug can be fixed later. Other, more serious bugs take priority

Medium: Bug can be fixed in the normal course of development and testing.

High: Bug must be resolved at the earliest as it affects the system adversely and renders it unusable until it is resolved.

5 Bug life cycle



5 Bug life cycle

New: When a new defect is logged and posted for the first time. It is assigned a status as NEW.

Assigned: Once the bug is posted by the tester, the lead of the tester approves the bug and assigns the bug to the developer team

Open: The developer starts analyzing and works on the defect fix

Fixed: When a developer makes a necessary code change and verifies the change, he or she can make bug status as “Fixed.”

Pending re-test: Once the defect is fixed the developer gives a particular code for retesting the code to the tester. Since the software testing remains pending from the testers end, the status assigned is “pending retest.”

Verified: The tester re-tests the bug after it got fixed by the developer. If there is no bug detected in the software, then the bug is fixed, and the status assigned is “verified.”

Re-open: If the bug persists even after the developer has fixed the bug, the tester changes the status to “reopened”. Once again, the bug goes through the life cycle.

Closed: If the bug is no longer exists then tester assigns the status “Closed.”

Duplicated: If the defect is repeated twice or the defect corresponds to the same concept of the bug, the status is changed to “duplicate.”

Invalid: If the developer feels the defect is not a genuine defect, then it changes the defect to “invalid.”

Deferred: If the present bug is not of a prime priority and if it is expected to get fixed in the next release, then status “Deferred” is assigned to such bugs

Not a bug: If it does not affect the functionality of the application then the status assigned to a bug is “Not a bug”.

6 Bug tracking tools

Jira is a commercial tool that is used for bug tracking, project management, and issue tracking in manual testing.
Jira includes different features like reporting, recording, and workflow. In Jira, we can track all kinds of bugs and issues, which are related to the software and generated by the team.

Bugzilla is an open-source bug tracking tool, the customers can report their issues through a portal.
It is also used as a test management tool, so you can create test cases and run them to report results.

Redmine is another open-source tool, it's used to track the issues and web-based project management tool. Redmine tool is written in Ruby programing language and compatible with multiple databases like MySQL, Microsoft SQL, and SQLite.

Mantis Bug Tracker is a web-based open-source bug tracking system. It is written in PHP.

Backlog is widely used to manage the IT projects and track the bugs. It is mainly built for the development team for reporting the bugs with the complete details of the issues, comments. Updates and change of status. It is a project management software.

Micro Focus Quality Center formerly known as **HP Quality Center** is a quality management software. It includes requirements management, test management and business process testing for IT and application environments.

GitHub and **GitLab** are code hosting platforms for version control and collaboration. Both platforms have a feature to handle project issues, so you can track bugs as a kind of issue.

6 Bug tracking tools

Bug ID	Every bug or defect has its unique identification number
Description	This includes the abstract of the issue.
Steps to reproduce	This includes the detailed steps of the issue with the screenshots attached so that developers can recreate it.
Product Version	This includes the product version of the application in which the defect is found.
Component	This includes the part of the product where this bug seems to be affected
Date raised	This includes the Date when the bug is reported
Date resolved	This includes the Date when the bug is fixed
Date verified	This includes the Date when the bug-fix is verified
Reporter	This includes the details of the tester who reported the bug like Name and ID
Status	This field includes the Status of the defect like New, Assigned, Open, Verified, Closed, Failed, Deferred, etc.
Assignee	This field includes the details of the developer who fixed it like Name and ID
Severity	Based on the severity (Critical, Major or Minor) it tells us about impact of the defect or bug in the software application
Priority	Based on the Priority set (High/Medium/Low) the order of fixing the defect can be made

Bugzilla Demo